

8 Comparison Tables for Exam

8.1 Optimizer Comparison

Optimizer	Momentum?	Adaptive LR?	Bias Corr?	Best For
SGD	No	No	–	Basic tasks
SGD+Momentum	Yes	No	–	Vision, best accuracy
Adagrad	No	Yes (sum)	No	Sparse/NLP (but LR dies)
RMSprop	No	Yes (EMA)	No	RNNs
Adam	Yes	Yes (EMA)	Yes	Default choice

8.2 Adagrad vs RMSprop

	Adagrad	RMSprop
Gradient accumulation	Sum (only grows)	EMA (forgets old)
Learning rate over time	Vanishes to 0	Stays healthy
Long training	Fails	Works
Use case	Short training, sparse	RNNs, deep nets

8.3 Adam vs SGD+Momentum

	Adam	SGD+Momentum
Ease of use	Easy (good defaults)	Needs tuning
Training speed	Faster	Slower
Final accuracy	Good	Often better
Generalization	Can overfit more	Usually better
Best for	Prototyping, NLP	Production, Vision

8.4 Xavier vs He Initialization

	Xavier	He
Activation	Tanh, Sigmoid, Linear	ReLU, LeakyReLU
Variance	$1/n_{in}$ or $2/(n_{in} + n_{out})$	$2/n_{in}$
Why this variance?	Symmetric activations	ReLU kills half signal
Modern usage	Less common	Very common

8.5 Batch Size Comparison

	Full Batch	Mini-Batch	SGD (1)
Batch size	All data	32-256	1
Gradient accuracy	Perfect	Good	Very noisy
Speed	Slow	Fast	Fastest
Memory	High	Medium	Low
Escape local min	Hard	Possible	Easy
GPU efficiency	Good	Good	Poor
Used in practice	Rarely	Always	Rarely

8.6 LR Scheduling Comparison

Schedule	How It Works	When to Use
Constant	Never changes	Baseline
Step Decay	Drop by factor every N epochs	Simple, common
Exponential	Smooth continuous decay	Gradual
Cosine	Follows cosine curve	State-of-the-art
Warm-up	Start low, increase	Large batch, Transformers

8.7 Gradient Problems Comparison

	Vanishing	Exploding
What happens	Gradient $\rightarrow 0$	Gradient $\rightarrow \infty$
Symptom	Early layers don't learn	NaN values
Cause	Sigmoid/tanh, deep nets	Large weights
Solutions	ReLU, skip connections, He init	Gradient clipping

Written Q1

Explain Adam's three components and why each is necessary.

Answer:

1. **First moment (momentum):** EMA of gradients. Smooths update direction, accelerates in consistent directions, dampens oscillations.
2. **Second moment (RMSprop):** EMA of squared gradients. Provides per-parameter adaptive learning rates.
3. **Bias correction:** Divides by $(1 - \beta^t)$. Fixes initialization bias since $m_0 = 0$, $v_0 = 0$ cause early estimates to be too small.

Written Q2

Compare Xavier and He initialization.

Answer:

- **Xavier:** For symmetric activations (tanh, sigmoid). Uses variance $= 1/n_{in}$. Assumes outputs are symmetric around zero.
- **He:** For ReLU. Uses variance $= 2/n_{in}$. The extra factor of 2 compensates for ReLU zeroing negative values (halving variance).

Written Q3

Explain vanishing and exploding gradients and their solutions.

Answer:

- **Vanishing:** Gradients shrink through layers (product of small values). Early layers don't learn. Solutions: ReLU, skip connections, proper initialization, batch norm.
- **Exploding:** Gradients grow through layers. Causes NaN values. Solutions: Gradient clipping, proper initialization, lower learning rate.

Written Q1

Explain sparse connectivity and parameter sharing in CNNs.

Model Answer:

Sparse Connectivity: Each output neuron connects to only a local region of the input (the receptive field), not all inputs. For a 3×3 filter, each output depends on only 9 input values. This dramatically reduces parameters compared to fully connected layers where every output connects to every input.

Parameter Sharing: The same filter weights are used at every spatial position in the image. Instead of learning separate edge detectors for each location, one filter is slid across the entire image. This means a 3×3 filter has only 9 parameters regardless of image size.

Together, these reduce parameters from millions (FC) to thousands (Conv), making training feasible and preventing overfitting.

Written Q2

Calculate the output size and number of parameters for a conv layer with: input $64 \times 64 \times 3$, 32 filters of 5×5 , padding=2, stride=1.

Model Answer:

Output spatial size:

$$W_{out} = \frac{64 - 5 + 2(2)}{1} + 1 = \frac{64 - 5 + 4}{1} + 1 = 64$$

Output dimensions: $64 \times 64 \times 32$

Parameters:

$$\begin{aligned} \text{Params} &= (K \times K \times D_{in} + 1) \times N_{filters} \\ &= (5 \times 5 \times 3 + 1) \times 32 = 76 \times 32 = 2,432 \text{ parameters} \end{aligned}$$

Written Q3

Explain the difference between translation equivariance and translation invariance.

Model Answer:

Translation Equivariance: If the input shifts by some amount, the output shifts by the same amount. Convolution is equivariant: if a feature moves 5 pixels right in the input, its detection also moves 5 pixels right in the activation map. Mathematically: $\text{Conv}(\text{Shift}(x)) = \text{Shift}(\text{Conv}(x))$

Translation Invariance: The output remains unchanged when the input shifts (within limits). Achieved through pooling, which aggregates over local regions. A small shift doesn't change the max value in a window, so the output stays the same.

CNNs combine both: equivariant convolutions (detect features anywhere) followed by invariant pooling (classify regardless of position).

Written Q4

Describe the four transfer learning scenarios based on data size and domain similarity.

Model Answer:

1. Small Data + Similar Domain: Freeze all conv layers, train only the final classifier. Early features transfer directly.

2. Large Data + Similar Domain: Fine-tune the entire network with a lower learning rate for early layers.

3. Small Data + Different Domain: Hardest case. Try using features from earlier layers (more generic). Heavy data augmentation helps.

4. Large Data + Different Domain: Fine-tune the entire network. May need to train from scratch if domain is very different.

Written Q5

Compare 1×1 , transposed, and dilated convolutions.

Model Answer:

1×1 Convolution:

- Changes channel depth, not spatial size
- Used for dimensionality reduction (bottleneck)
- Adds cross-channel interaction and non-linearity

Transposed Convolution:

- Increases spatial dimensions (upsampling)
- Used in segmentation decoders, GANs
- Learnable upsampling

Dilated Convolution:

- Keeps spatial size, increases receptive field
- Inserts gaps between filter elements
- Used when you need large receptive field without losing resolution

15 Comparison Tables

15.1 MLP vs CNN

Aspect	MLP	CNN
Connectivity	Fully connected	Sparse (local)
Parameter sharing	No	Yes (same filter everywhere)
Parameters for images	Millions	Thousands
Spatial structure	Lost (flattened)	Preserved
Translation	Not invariant	Equivariant (+ invariant with pool)

15.2 Padding Types

Type	P Value	Output Size	Use Case
Valid	0	Shrinks	Discard edges
Same	$(K - 1)/2$	Same as input	Preserve size
Full	$K - 1$	Larger	Every input contributes

15.3 Max vs Average Pooling

	Max Pooling	Average Pooling
Operation	Take maximum	Take average
Signal	Strongest activation	Overall presence
Use in hidden layers	Common	Rare
Use at output	Rare	Global Average Pooling

15.4 Special Convolution Types

Type	Spatial Effect	Depth Effect	Use
1×1	Same	Changes	Bottleneck
Transposed	Increases	Same/Changes	Upsampling
Dilated	Same	Same	Large receptive field

15.5 Transfer Learning Scenarios

	Similar Domain	Different Domain
Large Data	Fine-tune all	Fine-tune all
Small Data	Freeze early layers	Use earlier features

Written Q1

Explain how skip connections in ResNet solve the degradation problem.

Model Answer:

The degradation problem occurs when deeper networks perform **worse** than shallower ones, even on training data. This is not overfitting but optimization difficulty.

Skip connections solve this through:

1. **Direct gradient flow:** Gradients bypass conv layers and flow directly through skip connections, preventing vanishing gradients.
2. **Easy identity learning:** Instead of learning $H(x) = x$ (hard), network learns $F(x) = 0$ (easy). Output $H(x) = F(x) + x$ automatically becomes identity if $F(x) = 0$.
3. **Residual learning:** Each block only needs to learn small refinements (residuals), not complete transformations from scratch.
4. **Implicit ensemble:** Skip connections create paths of different lengths, acting like an ensemble of networks.

This enables training of 100+ layer networks that were previously impossible.

Written Q2

Compare the R-CNN, Fast R-CNN, and Faster R-CNN architectures.

Model Answer:

R-CNN (2014):

- Uses Selective Search for ~2000 region proposals
- Runs CNN **separately** on each proposal
- Very slow: ~47 seconds per image
- Bottleneck: Repeated CNN computation

Fast R-CNN (2015):

- Runs CNN **once** on the whole image
- Uses **ROI Pooling** to extract features for each proposal
- 10× faster: ~2 seconds per image
- Bottleneck: Selective Search still slow

Faster R-CNN (2015):

- Replaces Selective Search with learned **Region Proposal Network (RPN)**
- Uses **anchor boxes** at multiple scales/ratios
- End-to-end training of all components
- Nearly real-time: 0.2 seconds per image

Each version eliminates the bottleneck of the previous one.

Written Q3

Explain the Inception module and the role of 1×1 convolutions.

Model Answer:

The Inception module processes inputs at **multiple scales simultaneously**:

Structure:

- Four parallel paths: 1×1 , 3×3 , 5×5 convolutions, and max pooling
- All outputs are **concatenated** along the channel dimension
- Network learns which scales are important for each feature

Role of 1×1 convolutions:

1. **Dimensionality reduction:** Before expensive $3 \times 3/5 \times 5$ convolutions, 1×1 reduces channels (e.g., $256 \rightarrow 64$), dramatically cutting computation
2. **Add non-linearity:** Each 1×1 is followed by ReLU, adding representational power
3. **Cross-channel interaction:** 1×1 combines information across all channels at each spatial position

Without 1×1 bottlenecks, computational cost would be prohibitive. This is why GoogLeNet has only 5M parameters despite 22 layers (vs VGG's 138M).

Written Q4

Compare semantic segmentation with object detection.

Model Answer:

Aspect	Detection	Segmentation
Output	Bounding boxes + labels	Pixel-wise labels
Precision	Object-level	Pixel-level
Boundaries	Rectangular approximation	Exact object shape
Architecture	Faster R-CNN, YOLO	FCN, U-Net
Use case	"Find cars"	"Segment road from sky"

Key differences:

- Detection finds object locations with boxes; segmentation classifies every pixel
- Segmentation requires encoder-decoder architecture to restore resolution
- Detection uses region proposals; segmentation is naturally dense prediction

Written Q5

Explain depthwise separable convolutions and their efficiency.

Model Answer:

Depthwise separable convolutions break standard convolution into two operations:

1. Depthwise Convolution:

- One filter per input channel (M filters for M channels)
- Each filter only processes its own channel spatially
- Cost: $D_K^2 \cdot M \cdot D_F^2$

2. Pointwise Convolution (1×1):

- N filters of size $1 \times 1 \times M$
- Combines channels to create output depth
- Cost: $M \cdot N \cdot D_F^2$

Efficiency: Standard conv: $D_K^2 \cdot M \cdot N \cdot D_F^2$

Separable conv: $D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2$

Ratio: $\frac{1}{N} + \frac{1}{D_K^2}$

For 3×3 filters with 256 outputs: approximately **9× fewer operations** with similar accuracy. This enables efficient models like MobileNet for mobile devices.

12.1 CNN Architecture Evolution

Architecture	Year	Layers	Params	Key Innovation
VGG-16	2014	16	138M	3×3 filters only
GoogLeNet	2014	22	5M	Inception module + 1×1 bottleneck
ResNet-50	2015	50	25M	Skip connections
ResNet-152	2015	152	60M	Skip connections
DenseNet-121	2016	121	8M	Dense connections
MobileNet-v1	2017	28	4M	Depthwise separable conv

12.2 R-CNN Family Comparison

Method	Proposals	CNN Runs	Speed	Key Feature
R-CNN	Selective Search	2000/img	47s	Per-region CNN
Fast R-CNN	Selective Search	1/img	2s	ROI Pooling
Faster R-CNN	RPN (learned)	1/img	0.2s	End-to-end

12.3 ResNet vs DenseNet

	ResNet	DenseNet
Connection	Skip (add)	Dense (concatenate)
Feature combination	Element-wise sum	Channel stacking
Feature reuse	Partial	Maximum
Parameters	More	Fewer
Gradient flow	Through skips	Direct to all layers

12.4 Standard vs Depthwise Separable

	Standard Conv	Depthwise Separable
Operation	Spatial + channel together	Spatial then channel
# Filters	N filters of $D_K \times D_K \times M$	$M + N$ filters
Computation	$D_K^2 \cdot M \cdot N \cdot D_F^2$	$D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2$
Reduction	–	8-9× faster

12.5 Classification vs Detection vs Segmentation

Task	Output	Granularity	Example
Classification	1 label/image	Image-level	ResNet, VGG
Detection	Boxes + labels	Object-level	Faster R-CNN
Segmentation	Label/pixel	Pixel-level	FCN, U-Net

Written Q1: Explain Attention Mechanism (5 marks)

Question: Explain how the attention mechanism solves the bottleneck problem in seq2seq models.

Model Answer:

The bottleneck problem occurs because basic seq2seq models compress the entire input sequence into a single fixed-size context vector (1 mark).

Attention solves this by allowing the decoder to “look back” at all encoder hidden states at each decoding step (1 mark).

The mechanism works as follows:

1. Compute alignment scores between decoder state and each encoder hidden state
2. Apply softmax to get attention weights
3. Compute weighted sum of encoder states as the context (2 marks)

This creates a dynamic context vector that focuses on relevant parts of the input for each output token, effectively removing the fixed-size bottleneck (1 mark).

Written Q3: Multi-Head Attention (5 marks)

Question: Explain multi-head attention and why it is beneficial compared to single-head attention.

Model Answer:

Multi-head attention runs multiple attention operations in parallel, each with different learned projections (1 mark).

How it works:

- Input is projected into Q, K, V for each head using different weight matrices
- Each head computes attention independently
- Outputs are concatenated and projected (2 marks)

Benefits:

- Different heads can capture different types of relationships (syntactic, semantic, positional)
- Allows model to jointly attend to information from different representation subspaces
- Increases expressiveness without significantly increasing parameters (2 marks)

Written Q4: Positional Encoding (5 marks)

Question: Why do Transformers need positional encoding? Describe the sinusoidal approach.

Model Answer:

Transformers need positional encoding because self-attention is permutation-invariant – it treats input as a set without inherent order information (1 mark).

Sinusoidal Encoding: Uses sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

(2 marks)

Properties:

- Bounded values (between -1 and 1)
- Unique encoding for each position
- Can generalize to sequences longer than seen during training
- Allows model to learn relative positions (2 marks)

What is BERT?

BERT = Bidirectional Encoder Representations from Transformers

Key Innovation: Uses ONLY the encoder (no decoder), with bidirectional attention!

Pre-training Tasks:

1. **Masked Language Modeling (MLM):**

- Randomly mask 15% of tokens
- Model predicts masked tokens
- Forces model to use BOTH left and right context

2. **Next Sentence Prediction (NSP):**

- Given two sentences, predict if B follows A
- Helps understand sentence relationships